

LAPACK 可用求解器见: Available DRIVER routines

函数名中第一个字母代表计算精度:

- s: 代表单精度浮点数 (float)。
- d: 代表双精度浮点数 (double)。
- c: 代表复数单精度浮点数 (complex float)。
- z: 代表复数双精度浮点数 (complex double)。

1 变量解释

首先进行变量解释, 可以结合后面的例子理解

1.1 nrhs : number of right hand side

nrhs 是一个整数参数, 用于指定右手边 (Right Hand Side, 简称 RHS) 的列数。在解线性方程组时, **nrhs** 指定了右手边的列数, 即待求解方程组的个数。

例如, 对于一个 n 阶方阵 A , 我们要求解线性方程组 $Ax = B$, 其中 B 是一个 n 行 k 列的矩阵 (每列是一个 RHS)。那么, 在调用 LAPACKE_dgesv 时, **nrhs** 应该设置为 k , 此时

$$B = \begin{bmatrix} | & | & & | \\ b_1 & b_2 & \cdots & b_k \\ | & | & & | \end{bmatrix}.$$

一般情况下, 即 $Ax = b$ 时, **nrhs** 为 1, 表示只有一个 RHS 需要求解。

1.2 lda / ldb : leading dimension

对于一个 m 行 n 列的矩阵, 它通常在内存中以连续的一维数组的形式存储。leading dimension 指定了在这个一维数组中, 相邻两行之间的间隔 (步长)。

在行主序存储 (Row-Major Order) 中, leading dimension 就是矩阵的列数 n 。每一行的数据在内存中是连续存放的, 相邻两行之间的间隔是 n 个元素。

在列主序存储 (Column-Major Order) 中, leading dimension 就是矩阵的行数 m 。每一列的数据在内存中是连续存放的, 相邻两列之间的间隔是 m 个元素。

1.3 ipiv

ipiv 是一个整数数组, 用于存储部分选主元的信息。在线性代数求解中, 选主元是高斯消元法中的一种策略, 用于避免在计算过程中出现数值不稳定或者误差过大的情况。

在 LAPACK 中的一些线性方程组求解函数（如 LAPACKE_dgesv）中，需要提供一个 `ipiv` 数组作为参数。`ipiv` 数组的长度应该与矩阵的维度相同（即与矩阵的行数或列数相同），并用于存储选主元的信息。

具体来说，`ipiv` 数组中存储的是一个排列，例如，如果 `ipiv[i] = j`，表示在第 i 步的消元过程中，第 i 行和第 j 行进行了交换。

2 使用一般矩阵求解器

配置好 LAPACK 环境后，使用 LAPACK 的 `dgesv` 函数求解 3×3 线性方程组。考虑 $Ax = b$ ，其中：

$$A = \begin{bmatrix} 2 & 1 & -1 \\ 3 & -2 & 4 \\ -1 & 3 & -2 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 4 \\ 3 \end{bmatrix}.$$

我们将使用 LAPACK 的 `dgesv` 函数来解决这个方程组。

2.1 运行成功代码

使用 LAPACK 的 `dgesv` 函数的代码如下：

```
#include <iostream>
#include <lapacke.h>

int main() {
    double A[9] = {2.0, 1.0, -1.0, 3.0, -2.0, 4.0, -1.0, 3.0, -2.0};
    double b[3] = {1.0, 4.0, 3.0};
    int n = 3; // 矩阵的维度
    int nrhs = 1; // 右手边的列数，即b的列数
    int lda = n; // A的leading dimension，即A矩阵的列数
    int ldb = 1; // B的leading dimension，即B矩阵的列数

    int ipiv[n]; // 存储部分选主元的信息
    int info; // 存储函数的返回值

    // 使用LAPACK的dgesv函数求解方程组
    info = LAPACKE_dgesv(LAPACK_ROW_MAJOR, n, nrhs, A, lda, ipiv, b, ldb);
    if (info != 0) {
        std::cout << "求解方程组失败" << std::endl;
        return 1;
    }
}
```

```

// 打印解
std::cout << b[0] << " " << b[1] << " " << b[2] << std::endl;

return 0;
}

```

可以看出 LAPACK 把解覆盖在 $\mathbf{b}$ 。上述矩阵储存的格式为行主序，若要使用列主序需要进行修改，其主要区别在于 $\mathbf{A}$ 和 $\mathbf{B}$ 储存矩阵的方式，为体现 $\mathbf{B}$ 的区别，将其设为

$$B = \begin{pmatrix} 1 & 2 \\ 4 & 5 \\ 3 & 6 \end{pmatrix}$$

```

double A[3*3] = {2.0, 3.0, -1.0, 1.0, -2.0, 3.0, -1.0, 4.0, -2.0}; //
    列主序存储
double B[3*2] = {1, 4, 3, 2, 5, 6}; // 列主序存储

int nrhs = 2;
int lda = n; // A的leading dimension, 即A矩阵的行数
int ldb = n; // B的leading dimension, 即B矩阵的行数

info = LAPACKE_dgesv(LAPACK_COL_MAJOR, n, nrhs, A, lda, ipiv, B, ldb); //
    列主序存储

```

如果想使用 $\mathbf{A}[\][\]$ 储存数据，有以下例子：

```

#include <iostream>
#include <lapacke.h>

int main() {
    double A[3][3] = {2.0, 1.0, -1.0, 3.0, -2.0, 4.0, -1.0, 3.0, -2.0};
    double b[3] = {1.0, 4.0, 3.0};
    int n = 3; // 矩阵的维度
    int nrhs = 1; // 右手边的列数, 即b的列数
    int lda = n; // A的leading dimension, 即A矩阵的列数
    int ldb = 1; // B的leading dimension, 即B矩阵的列数

    int ipiv[n]; // 存储部分选主元的信息
    int info; // 存储函数的返回值

    // 使用 *A, 若前面用 b[\ ][\ ]则使用 *b
    info = LAPACKE_dgesv(LAPACK_ROW_MAJOR, n, nrhs, *A, lda, ipiv, b, ldb);
    if (info != 0) {

```

```

        std::cout << "求解方程组失败" << std::endl;
        return 1;
    }

    // 打印解
    std::cout << b[0] << " " << b[1] << " " << b[2] << std::endl;

    return 0;
}

```

如果想使用 `std::vector` 储存数据，有以下例子：

```

#include <iostream>
#include <vector>
#include <lapacke.h>

int main() {
    std::vector<double> A = {2.0, 1.0, -1.0, 3.0, -2.0, 4.0, -1.0, 3.0,
        -2.0}; // 行主序存储
    std::vector<double> B = {1.0, 2.0, 4, 5.0, 3.0, 6.0}; // 两个RHS
        行主序存储
    int n = 3; // 矩阵的维度
    int nrhs = 2; // 两个RHS
    int lda = n; // A的leading dimension, 即A矩阵的列数
    int ldb = 2; // B的leading dimension, 即B矩阵的列数

    std::vector<int> ipiv(n); // 存储部分选主元的信息
    int info; // 存储函数的返回值

    info = LAPACKE_dgesv(LAPACK_ROW_MAJOR, n, nrhs, &A[0], lda, &ipiv[0],
        &B[0], ldb); // 行主序存储
    if (info != 0) {
        std::cout << "求解方程组失败" << std::endl;
        return 1;
    }

    // 打印解
    for (int i = 0; i < n*ldb; i+=ldb) {
        std::cout << "x" << i/ldb+1 << " = " << B[i] << ", y" << i/ldb+1 << "
            = " << B[i + 1] << std::endl;
    }

    return 0;
}

```

```
}
```

3 使用带状矩阵求解器

以下矩阵中空白部分为 0, 考虑 $Ax = b$, 其中

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} & & & \\ a_{21} & a_{22} & a_{23} & a_{24} & & \\ & a_{32} & a_{33} & a_{34} & a_{35} & \\ & & a_{43} & a_{44} & a_{45} & a_{46} \\ & & & a_{54} & a_{55} & a_{56} \\ & & & & a_{65} & a_{66} \end{pmatrix}$$

其压缩储存形式为

$$AB(KL + KU + 1 + i - j, j) = A(i, j) \quad \max(1, j - KU) \leq i \leq \min(N, j + KL).$$

此时 $KL = 1$ 为下对角线数, $KU = 2$ 为上对角线数。即

$$AB = \begin{pmatrix} & & & & & \\ & & a_{13} & a_{24} & a_{35} & a_{46} \\ & a_{12} & a_{23} & a_{34} & a_{45} & a_{56} \\ a_{11} & a_{22} & a_{33} & a_{44} & a_{55} & a_{66} \\ a_{21} & a_{32} & a_{43} & a_{54} & a_{65} & \end{pmatrix}.$$

4 运行成功代码

```
#include <iostream>
#include <lapacke.h>

int main() {
    double AB[5][6] = {
        {0, 0, 0, 0, 0, 0},
        {0, 0, 3, 3, 3, 3},
        {0, 2, 2, 2, 2, 2},
        {7, 7, 7, 7, 7, 7},
        {5, 5, 5, 5, 5, 0}
    };
    double b[6] = {1, 4, 3, 2, 1, 3};
    int n = 6; // 矩阵的维度
```

```

int kl = 1;
int ku = 2;
int nrhs = 1; // 右手边的列数, 即b的列数
int ldab = n; // A的leading dimension, 即A矩阵的列数
int ldb = 1; // B的leading dimension, 即B矩阵的列数

int ipiv[n]; // 存储部分选主元的信息
int info; // 存储函数的返回值

// 使用 *AB, 若前面用 b[][] 则使用 *b
info = LAPACKE_dgbsv(LAPACK_ROW_MAJOR, n, kl, ku, nrhs, *AB, ldab, ipiv,
    b, ldb );
if (info != 0) {
    std::cout << "求解方程组失败" << std::endl;
    return 1;
}

// 打印解
for (auto i = 0; i < 6; ++i) {
    std::cout << b[i] << "\n";
}

return 0;
}

```